

1. Rappel du projet

Mon projet consiste à développer une application web dédiée au hobby des aquariums d'eau douce. Le problème observé est que de nombreux débutants comme passionnés reçoivent des conseils incomplets ou erronés en animalerie, ce qui entraîne des pertes

financières, une mauvaise santé des poissons et parfois l'abandon du hobby. Les utilisateurs visés sont les débutants, les propriétaires d'aquariums réguliers, les aquascapers, ainsi que les amateurs souhaitant se lancer dans l'élevage ou améliorer leurs installations techniques.

La proposition de valeur repose sur une plateforme en deux volets :

- une partie vitrine regroupant des conseils fiables, structurés et basés sur mes 15 années d'expérience personnelle et 10 années professionnelles ;
- une partie applicative offrant un tchat pour des conseils personnalisés.

2. Diagramme Entité-Relation

Le Back-End implémentera l'API pour accéder aux objets suivants :

2.1. Utilisateurs

Le back-office permet d'administrer les utilisateurs inscrits sur la plateforme en distinguant les utilisateurs standards des administrateurs. Cette gestion vise principalement à identifier les acteurs du service et à contrôler qui est autorisé à produire du contenu ou à répondre aux messages.

2.2. Articles

Les articles constituent le cœur du contenu expert de la plateforme. Le back-office offre les fonctionnalités nécessaires pour créer, modifier et publier des articles de conseil, tout en gérant leur état (brouillon ou publié).

Les Articles sont liés aux fiches des poissons : un article peut faire référence à plusieurs types de poissons et il peut exister plusieurs articles parlant d'un certain type de poisson.

2.3. Poissons

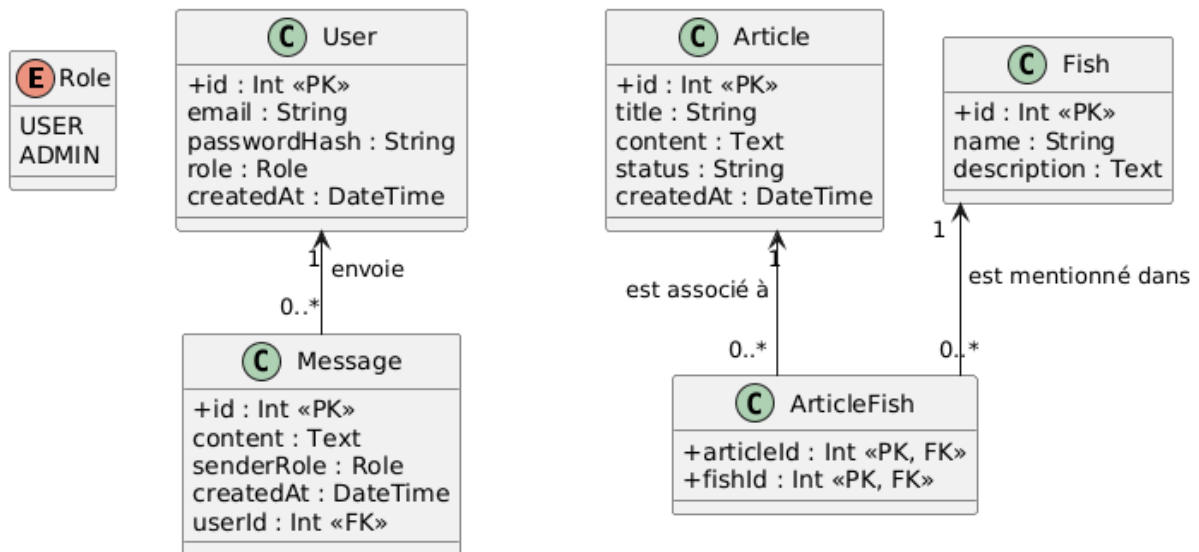
Les poissons représentent le cœur de l'expertise proposée par la plateforme. Chaque poisson correspond à une fiche descriptive regroupant des informations essentielles permettant de mieux comprendre ses besoins et ses particularités. Le back-office permet de gérer ces fiches ; les poissons peuvent ensuite être associés aux articles, ce qui permet de relier les conseils théoriques à des espèces concrètes rencontrées par les utilisateurs.

2.4. Messages

Le back-office permet de consulter et de gérer les messages échangés entre les utilisateurs et l'expert (administrateur) dans le cadre du service de tchat. Les messages sont associés aux utilisateurs et présentés de manière chronologique.

2.5. Diagramme

Le diagramme entité-relation associé est donné ci-dessous :



Comme il y a une liaison many-to-many entre les fiches des poissons et les articles, il est nécessaire d'avoir une table de liaison.

Description PlantUML

```
@startuml
skinparam classAttributeIconSize 0
skinparam linetype ortho

enum Role {
    USER
    ADMIN
}

class User {
    +id : Int <<PK>>
    email : String
    passwordHash : String
    role : Role
    createdAt : DateTime
}

class Article {
    +id : Int <<PK>>
    title : String
    content : Text
    status : String
    createdAt : DateTime
}

class Fish {
    +id : Int <<PK>>
    name : String
    description : Text
}

class Message {
    +id : Int <<PK>>
    content : Text
    senderRole : Role
    createdAt : DateTime
    userId : Int <<FK>>
}

class ArticleFish {
    +articleId : Int <<PK, FK>>
    +fishId : Int <<PK, FK>>
}

User "1" -- "0..*" Message : envoie
Article "1" -- "0..*" ArticleFish : est associé à
Fish "1" -- "0..*" ArticleFish : est mentionné dans
```

```

    content : Text
    senderRole : Role
    createdAt : DateTime
    userId : Int <<FK>>
}

class ArticleFish {
    +articleId : Int <<PK, FK>>
    +fishId : Int <<PK, FK>>
}

User "1" <-- "0..*" Message : « envoie »
Article "1" <-- "0..*" ArticleFish : « est associé à »
Fish "1" <-- "0..*" ArticleFish : « est mentionné dans »

@enduml

```

3.EndPoints

3.1. Authentification

- POST /auth/register : création d'un nouvel utilisateur
- POST /auth/login : authentification d'un utilisateur

Pour simplifier l'implémentation et se concentrer sur la logique métier et l'architecture backend, la gestion des sessions est volontairement simplifiée. L'identification de l'utilisateur est transmise explicitement dans les requêtes, sans mécanisme de session ou de token.

3.2. Utilisateurs (admin)

- GET /users
- GET /users/:id
- PUT /users/:id
- DELETE /users/:id

3.3. Poissons (front + admin)

- GET /fish (liste / recherche ?q=)
- GET /fish/:id
- POST /fish (admin)
- PUT /fish/:id (admin)
- DELETE /fish/:id (admin)

3.4. Articles (front + admin)

- GET /articles (liste + filtres ?q=&status=&fishId=)
- GET /articles/:id
- POST /articles (admin, avec fishIds[] à la création)
- PUT /articles/:id (admin)
- DELETE /articles/:id (admin)
- PUT /articles/:id/publish (admin)

3.5. Associations Article ↔ Fishs

Pour permettre de modifier les associations après la création.

- POST /articles/:id/fish/:fishId (ajouter)
- DELETE /articles/:id/fish/:fishId (retirer)

3.6. Messages (tchat asynchrone)

- GET /users/:id/messages (user/admin : lire l'historique)
- POST /users/:id/messages (user/admin : ajouter un message)
- Inbox admin (à répondre)
- GET /admin/messages/pending (liste des derniers messages "en attente")

4. Aquarium Backend API

URL de base : `http://localhost:3000`

4.1. Authentification

4.1.1. POST /auth/register

Objectif : Créer un compte utilisateur avec un rôle.

Entrée :

```
{
  "email": "user@aquarium.local",
  "password": "user123",
  "role": "USER"
}
```

Sortie (201) :

```
{
  "id": 3,
  "email": "user@aquarium.local",
  "role": "USER",
  "createdAt": "2025-02-01T12:00:00.000Z"
}
```

Codes :

201 : compte créé.
400 : email ou mot de passe manquant.
409 : email déjà utilisé.

4.1.2. POST /auth/login

Objectif : Valider les identifiants et retourner le profil utilisateur. Entrée :

```
{  
  "email": "admin@aquarium.local",  
  "password": "admin123"  
}
```

Sortie (200) :

```
{  
  "id": 1,  
  "email": "admin@aquarium.local",  
  "role": "ADMIN",  
  "createdAt": "2025-02-01T12:00:00.000Z"  
}
```

Codes :

200 : authentification réussie.
400 : email ou mot de passe manquant.
401 : identifiants invalides.

4.2. Utilisateurs (admin)

Toutes les routes exigent le header `x-user-id` d'un admin.

4.2.1. GET /users

Objectif : Lister tous les utilisateurs. Entrée : aucune. Sortie (200) :

```
[  
  {  
    "id": 1,  
    "email": "admin@aquarium.local",
```

```
"role": "ADMIN",  
  "createdAt": "2025-02-01T12:00:00.000Z"  
}  
]
```

Codes :

200 : liste retournée.
401 : authentification requise.
403 : accès admin requis.

4.2.2. GET /users/:id

Objectif : Récupérer un utilisateur par id. Entrée : paramètre de chemin :id.

Sortie (200) :

```
{  
  "id": 2,  
  "email": "user@aquarium.local",  
  "role": "USER",  
  "createdAt": "2025-02-01T12:00:00.000Z"  
}
```

Codes :

200 : utilisateur retourné.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : utilisateur introuvable.

4.2.3. PUT /users/:id

Objectif : Mettre à jour un utilisateur (profil ou rôle). Entrée :

```
{  
  "email": "updated@aquarium.local",  
  "password": "newpass",  
  "role": "USER"
```

```
}
```

Sortie (200) :

```
{
  "id": 2,
  "email": "updated@aquarium.local",
  "role": "USER",
  "createdAt": "2025-02-01T12:00:00.000Z"
}
```

Codes :

200 : utilisateur mis à jour.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : utilisateur introuvable.

4.2.4. DELETE /users/:id

Objectif : Supprimer un utilisateur (les messages sont supprimés en cascade). Entrée : paramètre de chemin :id. Sortie (204) : corps vide. Codes :

204 : utilisateur supprimé.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : utilisateur introuvable.

4.3. Poissons

4.3.1. GET /fish

Objectif : Lister les poissons, avec filtre optionnel par nom. Entrée (query) : ?q=neon. Sortie (200) :

```
[
  {
    "id": 1,
    "name": "Neon tetra",
    "description": "Petit poisson grégaire demandant une eau douce acide."
  }
]
```

```
]
```

Codes :

200 : liste retournée.

4.3.2. GET /fish/:id

Objectif : Récupérer un poisson par id. Entrée : paramètre de chemin :id. Sortie (200) :

```
{
  "id": 1,
  "name": "Neon tetra",
  "description": "Petit poisson grégaire demandant une eau douce acide."
}
```

Codes :

200 : poisson retourné.

400 : id invalide.

404 : poisson introuvable.

4.3.3. POST /fish (admin)

Objectif : Créer une fiche poisson. Entrée :

```
{
  "name": "Guppy",
  "description": "Poisson vif et facile pour débiter."
}
```

Sortie (201) :

```
{
  "id": 4,
  "name": "Guppy",
  "description": "Poisson vif et facile pour débiter."
}
```



```
}
```

Codes :

201 : poisson créé.
400 : données manquantes.
401 : authentification requise.
403 : accès admin requis.

4.3.4. PUT /fish/:id (admin)

Objectif : Mettre à jour une fiche poisson. Entrée :

```
{  
  "name": "Guppy",  
  "description": "Description mise à jour."  
}
```

Sortie (200) :

```
{  
  "id": 4,  
  "name": "Guppy",  
  "description": "Description mise à jour."  
}
```

Codes :

200 : poisson mis à jour.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : poisson introuvable.

4.3.5. DELETE /fish/:id (admin)

Objectif : Supprimer une fiche poisson. Entrée : paramètre de chemin :id. Sortie (204)

: corps vide. Codes :
204 : poisson supprimé.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.

404 : poisson introuvable.

409 : on veut enlever un poisson lié à des articles

4.4. Articles

4.4.1. GET /articles

Objectif : Lister les articles avec filtres optionnels. Entrée (query) : `?q=cycle&status=PUBLISHED&fishId=1`. Sortie (200) :

```
[
  {
    "id": 1,
    "title": "Démarrer un aquarium planté",
    "content": "Choisir un sol nutritif...",
    "status": "PUBLISHED",
    "createdAt": "2025-02-01T12:00:00.000Z",
    "authorId": 1,
    "fishes": [
      {
        "articleId": 1,
        "fishId": 1,
        "fish": {
          "id": 1,
          "name": "Neon tetra",
          "description": "Petit poisson grégaire..."
        }
      }
    ]
  }
]
```

Codes :

200 : liste retournée.

4.4.2. GET /articles/:id

Objectif : Récupérer un article et ses associations poisson. Entrée : paramètre de chemin `:id`. Sortie (200) : même format que dans la liste. Codes :

200 : article retourné.
400 : id invalide.
404 : article introuvable.

4.4.3. POST /articles (admin)

Objectif : Créer un article et associer des poissons. Entrée :

```
{  
  "title": "Guide de cyclage",  
  "content": "Lancer le cycle de l'azote...",  
  "status": "DRAFT",  
  "fishIds": [1, 2]  
}
```

Sortie (201) : article avec relations `fishes`. Codes :

201 : article créé.
400 : données manquantes.
401 : authentification requise.
403 : accès admin requis.

4.4.4. PUT /articles/:id (admin)

Objectif : Mettre à jour un article et, si fourni, remplacer les associations. Entrée :

```
{  
  "title": "Guide de cyclage",  
  "status": "PUBLISHED",  
  "fishIds": [1]  
}
```

Sortie (200) : article mis à jour avec `fishes`. Codes :

200 : article mis à jour.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : article introuvable.

4.4.5. PUT /articles/:id/publish (admin)

Objectif : Publier un article. Entrée : aucune. Sortie (200) : article mis à jour avec `status:`

`PUBLISHED`. Codes :

200 : article publié.

400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : article introuvable.

4.4.6. DELETE /articles/:id (admin)

Objectif : Supprimer un article. Entrée : paramètre de chemin :id. Sortie (204) : corps vide. Codes :

204 : article supprimé.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : article introuvable.

4.5. Associations Article ↔ Poisson

4.5.1. POST /articles/:id/fish/:fishId (admin)

Objectif : Associer un poisson à un article. Entrée : paramètres de chemin :id et :fishId.

Sortie (200) : article avec fishes mis à jour. Codes :

200 : association créée.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : article ou poisson introuvable.
409 : association déjà existante.

4.5.2. DELETE /articles/:id/fish/:fishId (admin)

Objectif : Retirer un poisson d'un article. Entrée : paramètres de chemin :id et :fishId.

Sortie (200) : article avec fishes mis à jour. Codes :

200 : association supprimée.
400 : id invalide.
401 : authentification requise.
403 : accès admin requis.
404 : association introuvable.

4.6. Messages

Toutes les routes exigent le header x-user-id de l'utilisateur cible ou d'un admin.

4.6.1. GET /users/:id/messages

Objectif : Récupérer l'historique du tchat d'un utilisateur. Entrée : paramètre de chemin :id. Sortie (200) :

```
[
  {
    "id": 1,
```

```
"content": "Bonjour...",
"senderRole": "USER",
"createdAt": "2025-02-01T12:00:00.000Z",
"userId": 2
}
]
```

Codes :

200 : messages retournés.
400 : id invalide.
401 : authentification requise.
403 : accès interdit.
404 : utilisateur introuvable.

4.6.2. POST /users/:id/messages

Objectif : Ajouter un message en tant qu'utilisateur ou admin. Entrée :

```
{
  "content": "Bonjour, j'ai une question..."
}
```

Sortie (201) :

```
{
  "id": 3,
  "content": "Bonjour, j'ai une question...",
  "senderRole": "USER",
  "createdAt": "2025-02-01T12:00:00.000Z",
  "userId": 2
}
```

Codes :

201 : message créé.
400 : contenu manquant ou id invalide.
401 : authentification requise.

403 : accès interdit.

404 : utilisateur introuvable.

4.7. Inbox admin

4.7.1. GET /admin/messages/pending (admin)

Objectif : Lister le dernier message par utilisateur lorsque le dernier message vient d'un utilisateur. Entrée : aucune. Sortie (200) :

```
[
  {
    "id": 5,
    "content": "Besoin d'aide...",
    "senderRole": "USER",
    "createdAt": "2025-02-01T12:00:00.000Z",
    "userId": 2,
    "user": {
      "id": 2,
      "email": "user@aquarium.local",
      "role": "USER",
      "createdAt": "2025-02-01T12:00:00.000Z"
    }
  }
]
```

Codes :

200 : liste retournée.

401 : authentification requise.

403 : accès admin requis.

5. Organisation du Projet node.js

5.1. Organisation du repertoire Prisma

Ce dossier regroupe la configuration Prisma, le client d'accès a la base et les migrations associées.

5.1.1. Fichiers

- `client.ts` : initialise et exporte une instance unique de `PrismaClient` pour être réutilisée dans l'application.
- `schema.prisma` : schéma Prisma (`datasource`, `generator`, `enums` et `models`) qui définit la structure de la base.
- `seed.ts` : script de peuplement des données de base (utilisateurs, poissons, articles, messages) et nettoyage préalable.

5.1.2. Dossier migrations/

Ici, "migration" désigne une évolution contrôlée du schéma de la base de données. Chaque migration capture une étape (créations de tables, ajout de colonnes, relations, indexes, etc.) et permet de reproduire le schéma de façon fiable entre environnements. Elles sont générées par Prisma en dev (par exemple avec `prisma migrate dev`).

Les migrations sont organisées par dossiers horodatés. Chaque dossier contient un `migration.sql` qui applique les changements de schema.

- `migrations/20251223204206_init/migration.sql` : migration initiale.
- `migrations/20251223214610_init_e/migration.sql` : migration suivante ajoutée après l'init.
- `migrations/migration_lock.toml` : verrou Prisma indiquant le provider et l'état des migrations.

5.1.3. Synthèse (questions fréquentes)

- Les migrations construisent le schéma de la base (structure) à partir de `schema.prisma`, pas les données.
- `seed.ts` remplit la base en données initiales, sans modifier la structure.
- En dev "jetable", on peut recréer la base via `prisma db push` sans utiliser les migrations.
- En production, les migrations restent importantes pour garder l'historique et appliquer les changements sans perte.
- Les outils Prisma sont lancés via `npm` (scripts `package.json`), pas par Express directement.
- `npm` fait partie de l'écosystème Node.js et sert à lancer ces scripts.

5.1.4. npm (contexte)

`npm` est le gestionnaire de paquets de Node.js. Il installe les dépendances et permet d'exécuter des commandes définies dans `package.json` (par exemple celles qui appellent Prisma). Il sert aussi à lancer des commandes via `npm run ...`, en se basant sur les scripts définis dans `package.json`. Les scripts à lancer doivent se trouver dans `node_modules/.bin`

6.A Expliquer

6.1. Mini charte – Paramètres d'une API REST

6.1.1. 1 Paramètres dans l'URL (path)

À utiliser pour identifier une ressource unique.

Exemples :

- GET /fish/12 → récupérer le poisson n°12
- PUT /articles/5 → modifier l'article n°5
- DELETE /users/3 → supprimer l'utilisateur n°3

👉 Règle : un :id dans l'URL = une ressource précise

6.1.2. 2 Paramètres de requête (query)

À utiliser pour filtrer, rechercher ou trier une liste.

Exemples :

- GET /fish?q=neon
- GET /articles?status=PUBLISHED
- GET /articles?fishId=4

👉 Règle : query = options sur une collection

6.1.3. 3 Corps de requête (JSON)

À utiliser pour envoyer des données métier lors d'une création ou d'une mise à jour.

Exemples :

- POST /fish
- { "name": "Guppy", "description": "Poisson facile" }
- PUT /articles/5
- { "title": "Nouveau titre", "content": "..." }

👉 Règle : JSON = données à enregistrer